Non-Linear Filtering for Target Tracking

Mathematical Foundations of the IMM-CKF Architecture

Huiyuan Zang

Advanced Concepts, EDGE Group

First Edition
June 2026

First Edition: June 2026

Contents

List of Tables

List of Figures

Preface

The Kalman Filter (KF) is arguably the most important algorithm in modern estimation theory. At its core, it is an elegant mathematical framework designed to extract the true state of a system from a series of incomplete and noisy measurements. In a perfectly linear world with Gaussian noise, the standard KF is the optimal estimator.

However, modern tactical tracking systems do not operate in a linear world.

In Real-World Vision-and-Telemetry-Based application, tracking has traditionally relied on 2D pixel-space bounding boxes generated by visual sensors. While sufficient for rudimentary camera-following, a 2D bounding box is physically meaningless for a real-world multi-sensor fusion system because pixels lack physical scale. A drone at 100 meters and an airliner at 10,000 meters can occupy the exact same number of pixels. To estimate actual target kinematics—true velocity, physical size, and metric trajectory—the tracking system must project 2D detections into a 3D Earth-fixed coordinate system, such as a North-East-Down (NED) frame.

This projection is where textbook mathematics collide with engineering reality. Mapping a 2D pixel backwards through nested Euler rotation matrices (representing the Gimbal, Mount, and UAV chassis) and a pinhole camera's perspective divide introduces severe mathematical nonlinearities.

The Evolution of Non-Linear Filtering

To handle systems where the measurement function is non-linear, engineers traditionally turn to variations of the Kalman Filter, though each comes with distinct failure modes in high-stress environments:

- **The Extended Kalman Filter (EKF)**: The EKF attempts to linearize the system by calculating the Jacobian (the first-order derivative) of the measurement function. However, deriving the analytical Jacobian of nested trigonometric rotation matrices coupled with perspective division is computationally fragile. During high-agility target maneuvers, the EKF's Taylor series approximation frequently fails, leading to rapid linearization error and track divergence.
- **The Unscented Kalman Filter (UKF)**: The UKF avoids Jacobians by propagating a deterministic set of sigma points through the non-linear function. While an improvement, the UKF requires the manual tuning of scaling parameters. In higher-dimensional state spaces subjected to extreme measurement noise, the central sigma point can develop a negative weight. This frequently causes the state covariance matrix to lose positive definiteness, resulting in a catastrophic numerical crash (NaN output) on edge hardware.
- **The Cubature Kalman Filter (CKF)**: The CKF resolves these issues by utilizing a third-degree spherical-radial cubature rule. It requires zero manual tuning parameters and mathematically guarantees strictly positive weights for all propagated points. This ensures maximum numerical stability, making it the premier choice for visual coordinate projection.

The Need for the IMM Architecture

Even with a stable non-linear filter, a single kinematic model cannot track all target types. For instance, a tracker using a purely 3D model on a ground target will misinterpret inherent bounding-box pixel jitter as vertical velocity, causing the mathematical track to "drift" into the sky or sink underground. Conversely, a constrained 2D ground model will completely fail to track an airborne target.

Furthermore, modern visual AI sensors act as noise multipliers. Bounding boxes inherently vibrate frame-to-frame. If an engineer implements a standard 11-state Constant Acceleration (CA) model, this high-frequency pixel jitter is mathematically interpreted as violent, instantaneous acceleration. The filter amplifies this noise, causing the predicted target state to vibrate uncontrollably until the track is completely lost.

The Interacting Multiple Model (IMM) architecture solves this by running multiple discrete mathematical models in parallel (e.g., a Ground model enforcing a strict altitude constraint, and an Airborne Bearings-Only model). The IMM dynamically shifts probability weights between these models based on how accurately their internal predictions match the incoming visual measurements.

Engineering Example 1: The Domain Transition (Liftoff)

When a drone is stationary on the tarmac, the IMM assigns maximum probability weight to the constrained Ground model. The moment the drone takes off, the 2D bounding box moves vertically in the video frame. The Ground model, mathematically insisting the altitude is zero, registers a massive prediction error (Innovation), and its calculated likelihood collapses. Simultaneously, the Airborne model correctly predicts the vertical movement. Governed by a Markov transition matrix, the IMM instantly transfers the tracking weight to the Airborne model, seamlessly unlatching the target from the Earth without losing the tracking ID.

Engineering Example 2: Multispectral Sensor Toggling

Consider an interceptor toggling its payload from a high-resolution 1080p visible sensor to a 480p Infrared (IR) sensor. The physical target kinematics do not change, but the measurement uncertainty space is drastically altered. IR microbolometers suffer from "thermal blooming," where thermal inertia causes the heat signature of a fast-moving target to smear across pixels, resulting in severe bounding box fluctuation. To prevent the tracking filter from chasing this noise—which would cause mechanical whiplash in the gimbal—the IMM-CKF dynamically injects an inflated measurement covariance scalar during the hardware toggle, forcing the filter to momentarily distrust the visual edges and rely on its internal kinematic predictions.

This book bridges the gap between pure mathematics and these operational realities. We will build the IMM-CKF from the ground up, proving every equation, and translating those proofs into robust C++ architectures designed for strict Size, Weight, and Power (SWaP) constrained hardware.

Acknowledgments

Writing a book that bridges the gap between pure mathematical theory and strict engineering reality is not a solitary endeavor. It is built upon the shoulders of giants and refined through the patience of peers and family.

First, my deepest gratitude goes to the mathematicians and engineers who laid the foundation for modern state estimation. From Rudolf E. Kálmán's original masterpiece, to the pioneers of non-linear filtering, and the architects of the cubature rule—thank you. Your theoretical brilliance has allowed generations of engineers to pull order from chaos. I also want to thank the wider aerospace and software engineering community, whose relentless pursuit of robust tracking systems under extreme constraints inspired the pragmatic approach of this book.

I am immensely grateful to the reviewers, colleagues, and early readers who dedicated their time to comb through the manuscript. Translating high-level mathematics into production C++ code is a complex task, and your rigorous feedback, mathematical debugging, and code reviews were instrumental in catching errors and refining the proofs. Your dedication has made this a far more accurate and valuable guide. Any errors that remain are entirely my own.

Finally, and most importantly, I want to thank my family. To my wife, Jinjing: thank you for your unwavering support throughout this demanding process. You constantly encourage me, ground me, and push me to be a better person every single day. And to my daughter, Halley: you are the angel of my life. Your bright spirit and joy enlighten my world, reminding me of the true purpose behind the work we do. This book is as much yours as it is mine.

Part I: The Mathematical Engine Room

Author's Note: Building the Toolbox

If you are an engineer looking at the integrals, partial derivatives, and series expansions in the upcoming pages and feeling a sense of mathematical dread, take a breath. This first section of the book exists simply to establish your foundational "toolbox."

You do not need to perfectly memorize how to analytically solve a multivariate Jacobian or calculate a spherical-radial integral right now. In the later chapters on the Extended (EKF), Unscented (UKF), and Cubature Kalman Filters (CKF), we will pull these tools out one by one. When we do, we will provide complete, step-by-step mathematical proofs explaining exactly how and why they are used to process real sensor data, alongside the C++ architectures to implement them on edge hardware.

For now, as you read through these mathematical prerequisites, simply focus on understanding the fundamental purpose of each tool and why a tracking system might need it.

Chapter 1

Advanced Calculus & Optimization

Before a tracking system can estimate a target's state, it must understand how that state changes over time and how it relates to the sensors observing it. In linear systems, these relationships are simple algebraic multipliers. In the real world of visual sensors and maneuvering targets, these relationships are highly non-linear, requiring advanced calculus to break them down into computable components.

1.1 Partial Derivatives and the Jacobian Matrix

The Mathematical Concept

For a single-variable function $y = f(x)$, the derivative $f'(x)$ gives the rate of change. However, tracking filters operate on multi-dimensional state vectors (e.g., 3D position, velocity, and acceleration). For a multi-variable vector function $\mathrm{y} = \mathrm{h}(\mathrm{x})$, where $\mathrm{x} \in \mathbb{R}^n$ and $\mathrm{y} \in \mathbb{R}^m$, we must calculate the partial derivative of every output with respect to every input.

This forms the **Jacobian Matrix** (J):

$$J = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_m}{\partial x_1} & \cdots & \frac{\partial h_m}{\partial x_n} \end{bmatrix}$$

Application in Target Tracking (The EKF)

The Extended Kalman Filter (EKF) relies entirely on the Jacobian matrix. When a camera observes a drone, the measurement function $\mathrm{z} = \mathrm{h}(\mathrm{x})$ maps the target's 3D real-world coordinates into 2D pixel space. Because this involves perspective division and trigonometric rotations, it is highly non-linear. The EKF uses the Jacobian $H_k = \frac{\partial \mathrm{h}}{\partial \mathrm{x}}$ evaluated at the current state estimate to linearize the function, allowing the filter to project the state covariance matrix into the measurement space[2]:

$$S_k = H_k P_k H_k^T + R_k$$

The Engineering Dilemma

Deriving the analytical Jacobian for a nested camera projection matrix (UAV chassis $\rightarrow$ gimbal $\rightarrow$ camera pinhole) results in massive, computationally fragile trigonometric chains. On SWaP-constrained

edge devices, evaluating these long sine/cosine chains for every target, every frame, burns critical CPU cycles. Furthermore, if the target maneuvers rapidly, the first-order Taylor series approximation of the Jacobian fails, causing catastrophic linearization error and track divergence.

1.2 Numerical Integration and Cubature Rules

The Mathematical Concept

Because analytical Jacobians are dangerous in high-agility tracking, modern filtering seeks to avoid them entirely. Instead of linearizing the function, we want to calculate the true expected value of the non-linear function mathematically. This requires evaluating a Gaussian-weighted integral over the entire state space:

$$I(f) = \int_{\mathbb{R}^n} f(\mathrm{x}) \exp(-\mathrm{x}^T \mathrm{x}) d\mathrm{x}$$

Solving this integral analytically in real-time is impossible. The **Spherical-Radial Cubature** Rule provides a numerical approximation. It decomposes the multi-dimensional integral into a spherical integral (direction) and a radial integral (distance) [1]. By carefully selecting exactly $2n$ deterministic points (where n is the state dimension), we can approximate the true integral with third-degree accuracy [1].

Application in Target Tracking (The CKF)

This integral is the beating heart of the Cubature Kalman Filter (CKF). Instead of differentiating the camera projection model, the CKF generates a set of 3D points representing the current uncertainty, pushes those specific points through the true non-linear camera projection model, and calculates the variance of the output.

C++ Implementation: Cubature Point Generation

In production code, generating these points must be done without dynamic memory allocation to ensure deterministic execution times in the main tracking loop. We utilize the Eigen library, a staple in high-performance C++ engineering.

```
#include <Eigen/Dense>
#include <vector>
#include <cmath>

using namespace Eigen;

class CubatureMath {
public:
    // Generates 2n cubature points based on the state dimension (n)
    // x: Current state vector
    // P_sqrt: The lower triangular matrix from Cholesky decomposition of Covariance P
    static MatrixXd generateCubaturePoints(const VectorXd& x, const MatrixXd& P_sqrt) {
        int n = x.rows();
        int num_points = 2 * n;
```

```
        // Pre-allocate the matrix to avoid heap fragmentation in the main loop
        MatrixXd cubaturePoints(n, num_points);

        // The scaling factor for the spherical-radial rule is simply sqrt(n)
        double scale = std::sqrt(static_cast<double>(n));

        for (int i = 0; i < n; ++i) {
            // Positive axis points
            cubaturePoints.col(i) = x + scale * P_sqrt.col(i);
            // Negative axis points
            cubaturePoints.col(i + n) = x - scale * P_sqrt.col(i);
        }

        return cubaturePoints;
    }
};
```

1.3 Multivariate Taylor Series Expansion

The Mathematical Concept

In engineering mathematics, it is often easier to approximate a highly complex function rather than solve it perfectly. A Taylor series allows us to approximate any smooth non-linear function as an infinite sum of polynomial terms, based on its derivatives at a specific operating point.For a multi-dimensional state vector x evaluated near a known predicted state $\hat{\mathrm{x}}$, the multivariate Taylor series expansion of a non-linear measurement function $\mathrm{h}(\mathrm{x})$ is expressed as:

$$\mathrm{h}(\mathrm{x}) = \mathrm{h}(\hat{\mathrm{x}}) + J(\mathrm{x} - \hat{\mathrm{x}}) + \frac{1}{2!}(\mathrm{x} - \hat{\mathrm{x}})^T \mathcal{H}(\mathrm{x} - \hat{\mathrm{x}}) + \dots$$

Where: - J is the Jacobian matrix (the first-order derivative, representing the immediate linear slope). - $\mathcal{H}$ is the Hessian matrix (the second-order derivative, representing the curvature). - The $+ \dots$ represents an infinite series of increasingly higher-order derivatives.

Application in Target Tracking (The EKF Linearization)

In classic tracking, we absolutely require matrices to calculate and propagate covariance (uncertainty). The Taylor series is the mathematical bridge that attempts to force a non-linear world to behave linearly.

The Extended Kalman Filter (EKF) takes a harsh, pragmatic approach to the equation above: it truncates the Taylor series right after the first-order Jacobian term, completely discarding the Hessian and all higher-order derivatives. It assumes that $\mathrm{h}(\mathrm{x}) \approx \mathrm{h}(\hat{\mathrm{x}}) + J(\mathrm{x} - \hat{\mathrm{x}})$.

The Engineering Dilemma

This first-order Taylor series approximation only holds mathematically true if the actual target state x remains extremely close to our filter's prediction $\hat{\mathrm{x}}$. If a fixed-wing UAV is flying straight and level, this approximation works well enough.

However, if an evasive drone suddenly executes a violent, high-G split-S maneuver, the true spatial state diverges rapidly from the filter's linear prediction. As the distance between x and $\hat{x}$ grows, the ignored higher-order terms (the truncation errors) become massive. The EKF's first-order Taylor series approximation mathematically shatters, causing rapid linearization error and total track divergence. This fundamental geometric flaw is precisely why modern aerospace interceptors must abandon the EKF and migrate toward the UKF and CKF architectures, which do not rely on Taylor series truncation.

C++ Implementation: The Cost of Linearization

In an object-oriented tracking architecture, understanding the difference between the true non-linear evaluation and the Taylor series approximation dictates how you architect your measurement updates.

```
#include <Eigen/Dense>
#include <cmath>

using namespace Eigen;

class KinematicApproximation {
public:
    // 1. The True Non-Linear Measurement (e.g., Camera Projection)
    // In the CKF and UKF, we pass deterministic sigma/cubature points directly through this,
    // preserving the true curvature of the physics.
    static VectorXd trueNonLinearMeasurement(const VectorXd& x) {
        VectorXd z(2);
        // Example: highly non-linear perspective division and trigonometric rotation
        z(0) = std::sin(x(0)) / x(2);
        z(1) = std::cos(x(1)) / x(2);
        return z;
    }

    // 2. The First-Order Taylor Approximation (The EKF Approach)
    // h(x) ≈ h(xHat) + J * (x - xHat)
    static VectorXd linearApproximation(const VectorXd& xTrue, const VectorXd& xHat, const MatrixXd& Jac
        VectorXd zHat = trueNonLinearMeasurement(xHat);
        VectorXd deltaX = xTrue - xHat;

        // The EKF applies the Jacobian but completely ignores second-order
        // curvature (the Hessian), leading to catastrophic errors during rapid maneuvers.
        return zHat + (Jacobian * deltaX);
    }
};
```

Chapter 2

Linear Algebra for Systems

Tracking algorithms are fundamentally exercises in linear algebra. Every prediction, measurement, and update is a matrix operation. To deploy an IMM-CKF on a modern embedded architecture (such as an NVIDIA Jetson Orin NX), an engineer must not only understand the mathematical identities but also how matrix shapes and memory bandwidth impact the overall processing pipeline.

2.1 Vectors, matrices, and tensor basics.

The Mathematical Concept

A tracking system defines the physical world using column vectors. A state vector x $\in \mathbb{R}^n$ holds the target's kinematics (e.g., X, Y, Z positions and velocities). Matrices act as operators that transform these vectors. The transition matrix $F \in \mathbb{R}^{n \times n}$ projects the state forward in time, while the measurement matrix $H \in \mathbb{R}^{m \times n}$ maps the n-dimensional state space into an m-dimensional observation space.

Application in Target Tracking

The dimensionality of these vectors and matrices dictates the computational load of the filter. An 11-state Constant Acceleration (CA) model requires maintaining an 11×11 covariance matrix containing 121 elements. An 8-state Constant Velocity (CV) model requires an 8×8 matrix with 64 elements.

While theoretical texts freely expand state dimensions, embedded systems processing multispectral perception pipelines face strict limits on Tensor Core availability. Structuring matrices cleanly ensures cache-friendly memory access during high-frequency telemetry fusion.

2.2 Eigenvalues, eigenvectors, and matrix decompositions (Cholesky decomposition, Singular Value Decomposition).

The Mathematical Concept

In Bayesian filtering, uncertainty is modeled using the State Covariance Matrix, P. A fundamental rule of probability is that variance cannot be negative. Therefore, a valid covariance matrix must be Positive Definite. Mathematically, a symmetric matrix A is positive definite if its eigenvalues are all strictly greater than zero, ensuring:

$$x^T Ax > 0$$

Before we decompose the covariance matrix, we must understand what it physically represents. If the state vector x is the filter's "best guess" of where the target is, the State Covariance Matrix P is the filter's "uncertainty bubble" around that guess.

For an n-dimensional state vector, P is a symmetric $n \times n$ matrix:

1 **The Diagonals (Variances)**: The elements along the main diagonal (e.g., $P_{0,0}$, $P_{1,1}$) represent the variance (σ^2) of each individual state. If $P_{0,0}$ is the variance of the X-position, a large number means the filter is highly uncertain about the target's exact X coordinate.

2 **The Off-Diagonals (Covariances)**: The elements off the main diagonal (e.g., $P_{0,1}$, $P_{1,0}$) represent the correlation between two states. If a target is turning, its X-velocity and Y-velocity are mathematically linked. A high covariance means that if the filter updates its belief about the X-velocity, it must proportionally update its belief about the Y-velocity.

Geometrically, in 3D space, the P matrix defines an ellipsoid (a 3D oval). The diagonals dictate how large the oval is, and the off-diagonals dictate how the oval is tilted or rotated in space.

To generate deterministic cubature points, the CKF requires calculating the "square root" of the P matrix. Because P is symmetric and positive definite, we use Cholesky Decomposition. This algorithm factors the matrix into the product of a lower triangular matrix L and its transpose L^T:

$$P = LL^T$$

Deep Dive: The Mathematical Proof > > This chapter focuses on the application of Cholesky decomposition for edge hardware. However, understanding exactly why every symmetric, positive-definite matrix can be uniquely factored this way is a beautiful piece of linear algebra. If you are interested in the rigorous step-by-step mathematical derivation and the proof by induction, please refer to **Appendix: The Square Root of a Matrix and Cholesky Decomposition**.

The Engineering Dilemma

In pure mathematical simulations, covariance matrices remain perfectly positive definite. However, when deployed on embedded hardware, 32-bit floating-point truncation, asynchronous telemetry latency, and sudden injections of measurement noise can cause the P matrix to become slightly asymmetric or develop small negative eigenvalues. If this happens, a standard Cholesky solver will throw a NaN error and crash the tracking thread.

C++ Implementation: Robust Cholesky Decomposition

A production-grade tracking orchestrator must wrap the decomposition in a protective layer, enforcing symmetry and regularizing the diagonal to guarantee execution.

```
#include <Eigen/Dense>
#include <iostream>
#include <stdexcept>

using namespace Eigen;
```

```
// Computes the lower triangular matrix L of a covariance matrix P.
MatrixXd computeRobustCholesky(MatrixXd P) {
    // 1. Enforce strict mathematical symmetry to mitigate floating-point drift
    P = 0.5 * (P + P.transpose());

    // 2. Attempt standard Cholesky Decomposition
    LLT<MatrixXd> lltOfP(P);

    if (lltOfP.info() == Eigen::NumericalIssue) {
        // Engineering Fallback: Matrix lost positive definiteness.
        // Inject a tiny process noise (epsilon) along the diagonal
        // to force eigenvalues back into positive territory.
        double epsilon = 1e-6;
        MatrixXd PReg = P + MatrixXd::Identity(P.rows(), P.cols()) * epsilon;

        LLT<MatrixXd> lltOfPReg(PReg);
        if (lltOfPReg.info() == Eigen::Success) {
            return lltOfPReg.matrixL();
        } else {
            throw std::runtime_error("Cholesky decomposition failed post-regularization.");
        }
    }

    return lltOfP.matrixL();
}
```

2.3 Matrix inversion lemma (Woodbury identity) and proofs.

The Mathematical Concept

Inverting a matrix is one of the most computationally expensive operations in linear algebra ($O(n^3)$ complexity). The Woodbury matrix identity dictates that the inverse of a rank-k correction of a matrix can be computed by doing a rank-k correction to the inverse of the original matrix:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$$

Application in Target Tracking

The Kalman Gain (K) dictates how much the filter trusts the incoming visual measurement versus its own internal prediction. It is calculated as:

$$K = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

The term $(HP_{k|k-1}H^T + R)$ is the Innovation Covariance matrix (S). If we are fusing dense data arrays (where the measurement vector is massive), directly inverting S will cause pipeline stalls. The Woodbury identity allows us to mathematically rearrange the equation so we only invert matrices equal to the size of the state vector, rather than the measurement vector.

Engineering Example

Imagine an architecture utilizing an 8-state kinematic model (X, Y, Z, Vx, Vy, Vz, Width, Height) but processing an array of 50 simultaneous pixel threshold measurements from a segmented infrared target. Inverting a 50×50 matrix using standard LU decomposition on an embedded GPU/CPU complex is expensive. Using the Woodbury identity, the tracking engine can mathematically bypass the 50×50 inversion and instead invert an 8×8 matrix, maintaining a strict 60fps processing throughput.

Chapter 3

Differential Equations & Kinematics

While Chapter 2 provided the linear algebra tools to handle uncertainty, tracking algorithms ultimately need something physical to calculate. We must mathematically describe how a target moves through space.

The core challenge of tracking is a mismatch of domains: target physics (gravity, thrust, drag) operate continuously without interruption, but our tracking processors and visual sensors operate discretely, capturing snapshots of the world at 60 frames per second or processing telemetry at 10Hz. This chapter details how we model the continuous world and safely discretize it for embedded hardware.

3.1 Ordinary Differential Equations (ODEs) and continuous-time system modeling.

The Mathematical Concept

In kinematics, we describe a system's state using a vector $\mathrm{x}(t)$. To understand where the target will be in the future, we must model its rate of change over time. This is expressed as an Ordinary Differential Equation (ODE).

For a non-linear continuous-time dynamic system, the state derivative is defined as:

$$\dot{\mathrm{x}}(t) = f(\mathrm{x}(t), \mathrm{u}(t), \mathrm{w}(t))$$

Where: - $\dot{\mathrm{x}}(t)$ is the rate of change of the state vector (e.g., velocity is the derivative of position, acceleration is the derivative of velocity). - $f(\dots)$ is the non-linear function describing the physical laws governing the target. - $\mathrm{u}(t)$ is the known control input (e.g., autopilot commands, if tracking our own vehicle). - $\mathrm{w}(t)$ is the continuous-time process noise, capturing unknown disturbances like wind gusts or unmodeled target intent.

Application in Target Tracking

If we are tracking an uncooperative target—like an evasive drone—we do not have access to its control inputs $\mathrm{u}(t)$. We must rely entirely on our kinematic assumptions. The ODE simplifies to

$$\dot{\mathrm{x}}(t) = f(\mathrm{x}(t)) + \mathrm{w}(t)$$

For a simple 1D position (p) and velocity (v) model where velocity is assumed constant, the ODE system looks like this:

$$\dot{p} = v$$

$$\dot{v} = 0 + w$$

The Engineering Dilemma

Computers do not understand continuous time. An edge processor cannot evaluate an integral from $t = 0$ to $t = \infty$. It only understands arrays of memory evaluated at specific, discrete clock cycles. To predict a target's position at the time of the next camera shutter, we must numerically integrate the continuous ODE over a specific time step, Δt.

3.2 Discretization of continuous-time models (Runge-Kutta methods).

The Mathematical Concept

To convert continuous physics into discrete software updates ($\mathrm{x}_k \rightarrow \mathrm{x}_{k+1}$), we must approximate the integral of the ODE over the time interval Δt.

The simplest approach is the Euler method: $\mathrm{x}_{k+1} = \mathrm{x}_k + \Delta t \cdot f(\mathrm{x}_k)$. However, Euler integration assumes the rate of change remains completely flat over the entire Δt window. For high-speed targets executing turns, this assumption accumulates massive errors.Because the local error (error per step) is proportional to h^2, and the global error is proportional to h, therefore it relies entirely on a linear approximation, which means it drifts significantly if the true curve is highly non-linear.For stiff equations or highly dynamic systems, if the step size h is not infinitesimally small, the Euler method can become numerically unstable, causing the state estimates to diverge wildly.

The gold standard for aerospace tracking is the 4th-Order Runge-Kutta (RK4) method. RK4 evaluates the continuous derivative at four distinct points across the Δt window (the beginning, two at the midpoint, and one at the end) and computes a weighted average.

The RK4 update equations are:

$$\mathrm{k}_1 = f(\mathrm{x}_k)$$

$$\mathrm{k}_2 = f(\mathrm{x}_k + \frac{\Delta t}{2}\mathrm{k}_1)$$

$$\mathrm{k}_3 = f(\mathrm{x}_k + \frac{\Delta t}{2}\mathrm{k}_2)$$

$$\mathrm{k}_4 = f(\mathrm{x}_k + \Delta t\mathrm{k}_3)$$

$$\mathrm{x}_{k+1} = \mathrm{x}_k + \frac{\Delta t}{6}(\mathrm{k}_1 + 2\mathrm{k}_2 + 2\mathrm{k}_3 + \mathrm{k}_4)$$

Engineering Example When dealing with asynchronous sensor telemetry, Δt is rarely a static value. If a camera frame drops, the time since the last measurement might jump from 16ms to 32ms. Integrating over this variable Δt using Euler would cause the predicted bounding box to jitter wildly. Using RK4,

the tracking engine safely interpolates the curved kinematic trajectory regardless of the fluctuating time step, ensuring the predicted bounding box lands exactly where the visual AI expects it.

C++ Implementation: The RK4 Integrator

In our object-oriented architecture, we isolate the integrator so it can be used by any kinematic model. Because we are operating on edge hardware where cycle counts matter, we pass the derivative function as a lambda or std::function to keep the memory footprint tight.

```
#include <Eigen/Dense>
#include <functional>

using namespace Eigen;

class KinematicIntegrator {
public:
    // f: The continuous-time ODE function describing the physics
    // xVector: The current state vector
    // dt: The exact time elapsed since the last measurement
    static VectorXd rungeKutta4(const std::function<VectorXd(const VectorXd&)>& f,
                                const VectorXd& xVector,
                                double dt) {

        VectorXd k1 = f(xVector);
        VectorXd k2 = f(xVector + 0.5 * dt * k1);
        VectorXd k3 = f(xVector + 0.5 * dt * k2);
        VectorXd k4 = f(xVector);

        return xVector + (dt / 6.0) * (k1 + 2.0*k2 + 2.0*k3 + k4);
    }
};
```

3.3 Modeling constant velocity (CV), constant acceleration (CA), and coordinated turn (CT) dynamics.

Once we have our integrator, we must define the physical functions ($f(\mathrm{x})$) we intend to track. Within an Interacting Multiple Model (IMM) architecture, we run several of these concurrently.

3.3.1 The Constant Velocity (CV) Model

The CV model assumes the target travels in a straight line at a constant speed. Any changes in velocity (accelerations) are modeled purely as random process noise.

For visual tracking applications, we utilize an 8-state vector: $[X, Y, Z, V_x, V_y, V_z, W, H]^T$, where W and H are the physical width and height of the target.

In continuous time, the derivative function $f(\mathrm{x})$ is simply the velocities:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$

- $\dot{Z} = V_z$
- $\dot{V}_x = 0$
- $\dot{V}_y = 0$
- $\dot{V}_z = 0$
- $\dot{W} = 0$ (Target size is physically constant).
- $\dot{H} = 0$ (Target size is physically constant)

Because the CV model is perfectly linear, it does not strictly require RK4 integration. The discrete-time kinematic equations become:

$$X_{k+1} = X_k + V_{x,k}\Delta t$$
$$V_{x,k+1} = V_{x,k}$$
$$W_{k+1} = W_k$$
$$H_{k+1} = H_k$$

Mapping these equations across all variables yields our highly efficient, 8×8 discrete-time State Transition Matrix (F_{CV}):

$$F_{CV} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The Engineering Dilemma: The Lag vs. Noise Trade-off For a vision-and-telemetry-based tracker, the CV model is incredibly powerful because it naturally filters out the high-frequency "pixel jitter" generated by neural network bounding boxes. By refusing to explicitly calculate acceleration, the filter prevents the target's 3D state from vibrating wildly.

However, this mathematical stubbornness introduces a severe engineering dilemma: **Dynamic Lag**.

If an evasive target suddenly executes a sharp, high-G maneuver, the CV model mathematically assumes the target is still flying straight. It will confidently predict the target's next position along that straight line. By the time the visual measurement proves the target has turned, the prediction error (the Innovation) becomes massive. If the filter trusts its internal physics too much, the bounding box will lag behind the physical target, eventually separating entirely and causing a dropped track.

To solve this without reverting to the noisy Constant Acceleration model, engineers use a mathematical workaround: Process Noise Inflation. Within the IMM architecture, we deploy a secondary "High-Agility" CV model. This model uses the exact same F_{CV} matrix, but couples it with a massively inflated Process Noise Covariance matrix (Q). When a maneuver occurs, the IMM shifts probability to this high-agility model. The inflated Q matrix forces the Kalman Gain to mathematically distrust its own straight-line prediction and snap directly to the raw visual measurements until the turn is complete, completely eliminating dynamic lag without adding matrix dimensionality.

C++ Implementation

Because the CV model is the baseline state for the majority of the tracking lifecycle, its matrix generation must be incredibly lean. In C++, constructing this 8×8 identity matrix and injecting the Δt parameters is highly cache-friendly.

```cpp
#include <Eigen/Dense>

using namespace Eigen;

class KinematicModels {
public:
    // Generates the 8x8 State Transition Matrix for the Constant Velocity (CV) Model
    // dt: The time elapsed since the last update
    static MatrixXd getConstantVelocityMatrix(double dt) {
        // Initialize an 8x8 Identity matrix
        MatrixXd F = MatrixXd::Identity(8, 8);

        // Position updates linearly based on velocity (e.g., X = X + Vx * dt)
        F(0, 3) = dt;
        F(1, 4) = dt;
        F(2, 5) = dt;

        // Note: Velocities (indices 3, 4, 5) and Bounding Box dimensions
        // (indices 6, 7) inherently multiply by 1.0 from the Identity matrix,
        // representing their constant state.

        return F;
    }
};
```

3.3.2 The Constant Acceleration (CA) Model

The CA model introduces spatial acceleration states, expanding to an 11-state vector: $[X, Y, Z, V_x, V_y, V_z, A_x, A_y, A_z, W$

The ODE becomes:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$
- $\dot{Z} = V_z$
- $\dot{V}_x = A_x$
- $\dot{V}_y = A_y$
- $\dot{V}_z = A_z$
- $\dot{A}_x = 0$
- $\dot{A}_y = 0$
- $\dot{A}_z = 0$
- $\dot{W} = 0$ (Target size is physically constant)
- $\dot{H} = 0$ (Target size is physically constant)

Therefore, the kinematic equations are:

- Position: $X_{k+1} = X_k + V_{x,k}\Delta t + \frac{1}{2}A_{x,k}\Delta t^2$

- Velocity: $V_{x,k+1} = V_{x,k} + A_{x,k}\Delta t$
- Acceleration: $A_{x,k+1} = A_{x,k}$

Mapping these equations across all spatial axes yields our 11×11 discrete-time State Transition Matrix (F_{CA}):

$$F_{CA} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The Engineering Reality of CA: While theoretically rigorous for radar tracking ballistic missiles, the 11-state CA model is explicitly dangerous for vision-based tracking. Because visual AI bounding boxes suffer from high-frequency pixel jitter, an explicit acceleration state will interpret this jitter as violent, frame-to-frame G-forces. The filter will amplify this noise, causing the track to vibrate out of control. This is why our IMM architecture relies on an 8-state CV model with dynamically inflated process noise to simulate maneuvers, rather than explicitly tracking acceleration.

** C++ Implementation for the CA Model:**

```cpp
#include <Eigen/Dense>

using namespace Eigen;

class ConstantAccelerationModel {
public:
    // Generates the 11x11 State Transition Matrix for the CA Model
    static MatrixXd getTransitionMatrix(double dt) {
        MatrixXd F = MatrixXd::Identity(11, 11);

        double dt2Half = 0.5 * dt * dt;

        // Position updates based on velocity
        F(0, 3) = dt; F(1, 4) = dt; F(2, 5) = dt;

        // Position updates based on acceleration
        F(0, 6) = dt2Half; F(1, 7) = dt2Half; F(2, 8) = dt2Half;

        // Velocity updates based on acceleration
        F(3, 6) = dt; F(4, 7) = dt; F(5, 8) = dt;

        return F;
```

```
    }
};
```

3.3.3 The Coordinated Turn (CT) Model

The CT model is essential for tracking fixed-wing aircraft, which cannot simply slide sideways in the air; they must bank to turn. This introduces a Turn Rate parameter (ω). Assume that the tracking architecture relies on visual AI bounding boxes, you must also maintain the physical width and height of the target to utilize the scale prior. Adapting the CT model for your specific IMM-CKF pipeline results in a 9-state vector:

$$\mathrm{x} = [X, Y, Z, V_x, V_y, V_z, \omega, W, H]^T$$

- Z, V_z: Altitude and vertical velocity.
- W, H: Physical width and height of the target.

The standard CT model assumes the aircraft is banking to execute a turn in the horizontal plane ($X-Y$). Therefore, the turn rate ω strictly cross-couples and rotates the horizontal velocities (V_x and V_y). The vertical trajectory (Z-axis) is considered mathematically decoupled from the horizontal turn.

This model is highly non-linear. The ODE for a 3D horizontal turn is:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$
- $\dot{Z} = V_z$
- $\dot{V}_x = -\omega V_y$
- $\dot{V}_y = \omega V_x$
- $\dot{V}_z = 0$
- $\dot{\omega} = 0$
- $\dot{W} = 0$
- $\dot{H} = 0$

Because the turn rate (ω) multiplies with the velocities (V_x and V_y), the Coordinated Turn model is non-linear. Therefore, a static F_{CT} matrix of pure constants does not exist. (In an EKF, you would derive the Jacobian matrix here, but because we are building a CKF, we do not need to linearize it).

Instead, we directly evaluate the closed-form discrete transition function $\mathrm{x}_{k+1} = f(\mathrm{x}_k, \Delta t)$. The exact geometric equations for a target banking in the horizontal plane over time Δt are:

$$X_{k+1} = X_k + \frac{V_{x,k}}{\omega_k}\sin(\omega_k \Delta t) - \frac{V_{y,k}}{\omega_k}(1 - \cos(\omega_k \Delta t))$$

$$Y_{k+1} = Y_k + \frac{V_{x,k}}{\omega_k}(1 - \cos(\omega_k \Delta t)) + \frac{V_{y,k}}{\omega_k}\sin(\omega_k \Delta t)$$

$$V_{x,k+1} = V_{x,k}\cos(\omega_k \Delta t) - V_{y,k}\sin(\omega_k \Delta t)$$

$$V_{y,k+1} = V_{x,k}\sin(\omega_k \Delta t) + V_{y,k}\cos(\omega_k \Delta t)$$

(Note: Z, V_z, ω, W, and H remain linearly constant or are integrated straight forwardly).

The Engineering Dilemma (Division by Zero): Notice that the equations for X and Y require dividing by ω. If the aircraft stops turning and flies straight, ω approaches 0, which will cause a divide-by-zero hardware exception (NaN) and crash your filter. To solve this, your C++ implementation must check if ω is near zero, and if so, safely fall back to the standard Constant Velocity calculation for that frame.

C++ Implementation for the CT Model: In the CKF, you will pass your 18 cubature points (2×9 states) through this exact function during the Prediction Step.

```cpp
#include <Eigen/Dense>
#include <cmath>

using namespace Eigen;

class CoordinatedTurnModel {
public:
    // Evaluates the non-linear discrete state transition for the CT model.
    // xState: The 9-state vector [X, Y, Z, Vx, Vy, Vz, omega, W, H]^T
    static VectorXd evaluateTransition(const VectorXd& xState, double dt) {
        VectorXd xNext = xState; // Initialize with current state to carry over constants (Z, Vz, w, W, H

        double vx = xState(3);
        double vy = xState(4);
        double w = xState(6);

        // Z position updates linearly based on Vz
        xNext(2) = xState(2) + xState(5) * dt;

        // Engineering Check: Prevent divide-by-zero if the target is flying straight
        if (std::abs(w) < 1e-5) {
            // Degenerate to Constant Velocity model for X and Y
            xNext(0) = xState(0) + vx * dt;
            xNext(1) = xState(1) + vy * dt;

            // Velocities remain constant
            xNext(3) = vx;
            xNext(4) = vy;
        } else {
            // Apply the non-linear Coordinated Turn equations
            double sin_w_dt = std::sin(w * dt);
            double cos_w_dt = std::cos(w * dt);

            xNext(0) = xState(0) + (vx / w) * sin_w_dt - (vy / w) * (1.0 - cos_w_dt);
            xNext(1) = xState(1) + (vx / w) * (1.0 - cos_w_dt) + (vy / w) * sin_w_dt;

            xNext(3) = vx * cos_w_dt - vy * sin_w_dt;
            xNext(4) = vx * sin_w_dt + vy * cos_w_dt;
```

```
        }

        return xNext;
    }
};
```

Chapter 4

Probability Theory & Stochastic Processes

If the kinematics in Chapter 3 described how a target should move, probability theory describes how it actually moves in a chaotic universe. Sensors suffer from thermal noise, wind shears alter flight paths, and uncooperative targets execute unpredictable maneuvers. To build an optimal estimator, we cannot rely on deterministic certainties; we must compute with probabilities.

4.1 Random variables, probability density functions (PDFs), and Bayes' Theorem.

The Mathematical Concept

In tracking, the true kinematic state of a target (x) and the measurements from our sensors (z) are Continuous Random Variables. Because they can take on an infinite number of values (e.g., a drone can be at 100.0 meters, 100.001 meters, etc.), we cannot assign a probability to a specific, exact number.

Instead, we use a Probability Density Function (PDF), denoted as $p(\mathrm{x})$. The PDF defines the relative likelihood of the random variable falling within a particular spatial region.

In the context of aerospace tracking, the target's state x (where it actually is) and the measurement z (where the sensor thinks it is) are not single, absolute numbers. They are probability distributions—specifically, bell curves (Gaussians) representing uncertainty. Bayes' theorem is the mathematical engine that merges these conflicting curves into a single truth.

$$p(\mathrm{x} \mid \mathrm{z}) = \frac{p(\mathrm{z} \mid \mathrm{x})p(\mathrm{x})}{p(\mathrm{z})}$$

Where: - Prior $p(\mathrm{x})$:

The Engineering Meaning : This is the filter's Prediction.

What it is: Based on the kinematic ODEs we built in Chapter 3, this is where the filter believes the target is before the camera shutter opens.

The Shape: Because of process noise (wind, unknown target maneuvers), this prediction is not a single point; it is a Gaussian curve spread out over space. A wider curve means the filter is highly uncertain about its prediction.

- Likelihood $p(\mathrm{z} \mid \mathrm{x})$:

  The Engineering Meaning: This is the Sensor Measurement.

  What it is: This answers the question: "If the target is actually at state x, how likely is it that my camera would output the exact bounding box z that I just received?"

  The Shape: Visual AI sensors have inherent pixel jitter and calibration errors. Therefore, the measurement itself is a Gaussian curve centered on the pixel detection. A cheap, noisy sensor produces a very wide, flat curve; a military-grade laser rangefinder produces a very narrow, sharp curve.

- The Posterior: $p(\mathrm{x} \mid \mathrm{z})$

  The Engineering Meaning: This is the Updated Estimate (The final output of the filter for this frame).

  What it is: This is our new, refined belief of the target's true state, given both our internal physics prediction and the new sensor data.

  The Math: The numerator of Bayes' theorem simply multiplies the Prior curve and the Likelihood curve together.

- Evidence $p(\mathrm{z})$

  The Engineering Meaning: The Normalizing Constant.

  What it is:When you multiply two Gaussian curves together, the resulting curve shrinks. In probability theory, the total area under a PDF curve must always equal exactly 1.0 (representing 100% total probability). The Evidence term $p(\mathrm{z})$ calculates the total, absolute probability of receiving this specific measurement across the entire universe of possible states. By dividing by this term, we scale the new multiplied curve back up so its area equals 1.0.

Engineering Example: The 1D Drone Intercept

Imagine a 1D tracking scenario. We are tracking a drone's horizontal distance (X) along a runway.

1. The Prediction (Prior)

Our Constant Velocity (CV) model pushes the state forward in time. Based on the drone's previous speed, the filter predicts the drone is currently at 100 meters. Because a heavy crosswind was detected, our process noise is high. The filter's belief is a wide Gaussian curve centered at 100m.

2. The Measurement (Likelihood)

The camera processes the latest video frame and detects the drone. The bounding box centroid maps to a distance of 110 meters. The camera is known to have ± 5 meters of thermal noise. This creates a second, narrower Gaussian curve centered at 110m.

3. The Multiplication (Posterior)

We now have two conflicting pieces of data: the physics model says 100m, the camera says 110m. Bayes' theorem handles this conflict seamlessly by multiplying the two curves.

Because the camera's curve (Likelihood) is narrower (meaning it has less variance/uncertainty) than the physics prediction curve (Prior), the multiplication heavily favors the camera. The resulting Posterior curve will not peak perfectly in the middle at 105m. Instead, the mathematical peak will shift closer to the more "certain" data source—perhaps peaking at 108 meters.

Furthermore, because we have successfully fused two independent sources of information, the new Posterior curve will be narrower and taller than both the Prior and the Likelihood. Mathematically, Bayes' theorem guarantees that fusing data always reduces overall system uncertainty.

Deep Dive: The Birth of the Kalman Gain

We just stated that multiplying two Gaussian bell curves miraculously creates a third, perfectly shaped Gaussian curve. But why? And how does that relate to writing C++ tracking code?If you multiply the algebraic equations of these two curves, the resulting formula for the new Mean and Variance is exactly the 1D Kalman Filter Update Equation. For the step-by-step algebraic proof showing exactly how the Kalman Gain (K) is derived from this multiplication, refer to **Appendix: The Gaussian Multiplication Proof and the Origins of the Kalman Gain**.

The Engineering Dilemma (The Integration Problem)

The Evidence term $p(\mathrm{z})$ requires integrating the likelihood across the entire infinite state space: $p(\mathrm{z}) = \int p(\mathrm{z} \mid \mathrm{x}) p(\mathrm{x}) d\mathrm{x}$. For non-linear camera projections, this integral has no closed-form analytical solution. It is impossible to calculate on a CPU. This mathematical roadblock is exactly why the Cubature Kalman Filter uses numerical cubature points to approximate this integral, rather than attempting to solve Bayes' Theorem analytically.

4.2 Expectation algebra

To write tracking algorithms, we don't manipulate entire PDF curves; we manipulate their statistical properties using **Expectation Algebra**.

The Expected Value ($E[\mathrm{x}]$) is the mathematical center of mass of the PDF. In tracking, this is our state estimate, denoted as $\hat{\mathrm{x}}$ or μ.

The Covariance (P) measures how much the random variable is expected to spread out from its mean:

$$P = E[(\mathrm{x} - \hat{\mathrm{x}})(\mathrm{x} - \hat{\mathrm{x}})^T]$$

Expectation is a linear operator. This means that if we apply a linear matrix transformation to our state (like multiplying our state by the State Transition Matrix F), the expectation flows through cleanly.

(Note: The rigorous mathematical proof showing exactly how covariance propagates through linear matrices—resulting in the famous Kalman Filter equation $P_{k|k-1} = F P_{k-1} F^T + Q$—is fundamental to estimation theory. To keep this chapter focused on implementation, the complete derivation is provided in Appendix: Expectation Algebra and Covariance Propagation).

4.3 Gaussian distributions and their properties (multivariate normal distribution).

The Mathematical Concept

While Bayes' theorem theoretically works for any PDF shape, maintaining arbitrary, lopsided probability curves in software requires massive particle filters that choke embedded CPUs.

The Kalman Filter family makes a bold mathematical assumption: all uncertainties are Gaussian (Normal) distributions. Governed by the Central Limit Theorem, this is a safe assumption because the sum of many independent random noises (thermal jitter, wind, vibration) naturally tends toward a Gaussian shape.

A Multivariate Gaussian is entirely defined by just two parameters: its mean vector μ (the state estimate) and its covariance matrix P (the uncertainty). The PDF is evaluated as:

$$\mathcal{N}(\mathrm{x}; \mu, P) = \frac{1}{\sqrt{(2\pi)^n |P|}} \exp\left(-\frac{1}{2}(\mathrm{x}-\mu)^T P^{-1}(\mathrm{x}-\mu)\right)$$

Where:

- n is the dimension of the state vector.
- $|P|$ is the determinant of the covariance matrix.
- P^{-1} is the inverse of the covariance matrix.

The Engineering Connection to Chapter 2 Look closely at the Gaussian equation above. To evaluate a target's likelihood, the CPU must calculate the determinant $|P|$ and the inverse P^{-1}. If floating-point drift causes your covariance matrix to lose positive definiteness (as discussed in Chapter 2.2), the determinant $|P|$ becomes negative. The term $\sqrt{|P|}$ then attempts to take the square root of a negative number, resulting in an immediate software crash. This proves why rigorous Cholesky regularization is non-negotiable in production tracking.

4.4 Markov processes and transition probabilities.

The Mathematical Concept

A **Markov Process** (specifically a discrete-time Markov Chain) is a mathematical framework for modeling systems that transition from one state to another over time.

The defining rule of a **Markov Process** is the **Markov Property** (or "memoryless" property): The probability of transitioning to any future state depends entirely on the current state, and completely ignores the historical path taken to get there.

If a system has a set of possible states $S = \{s_1, s_2, \dots, s_n\}$, the Markov Property is written mathematically as:

$$P(X_{k+1} = s_j \mid X_k = s_i, X_{k-1} = s_h, \dots) = P(X_{k+1} = s_j \mid X_k = s_i)$$

These transition probabilities are organized into a square Transition Probability Matrix, denoted as Π (or P).

$$\Pi = [p_{ij}] \quad \text{where} \quad p_{ij} = P(M_j(k) \mid M_i(k-1))$$

For a 2-state system, it looks like this:

$$\Pi = \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix}$$

- p_{11} is the probability of staying in State 1.
- p_{12} is the probability of switching from State 1 to State 2.
- Because the system must be in one of the states at the next time step, every row in a Markov matrix must sum to exactly 1.0. ($p_{11} + p_{12} = 1.0$).

In the Interacting Multiple Model (IMM) architecture, we assume the target's decision to maneuver is a Markov Process. The target is in a specific kinematic "mode" M (e.g., M_1 = Constant Velocity, M_2 = High-Agility Turn).

Application in Target Tracking

In the IMM-CKF architecture, we are running multiple kinematic models simultaneously (e.g., a Constant Velocity (CV) model and a Coordinated Turn (CT) model). The IMM uses a Markov Process to govern how the filter switches between these physical models.

To see exactly how the Markov Transition Matrix (Π) controls the tracking filter, let's walk through a single frame of video (one time step, k) using concrete numbers.

0. The Initial State (Frame $k-1$):

In the previous frame, the drone was flying straight. Our filter was highly confident it was in CV mode

- Previous Probabilities: $\mu_1 = 0.80$ (80% CV), $\mu_2 = 0.20$ (20% CT).
- Previous State Estimates (Simplified to 1D Velocity for this example): $\hat{x}_1 = 100$ m/s, $\hat{x}_2 = 90$ m/s.

1. Defining the "Inertia" (Π Matrix)

As an engineer, you define the Transition Matrix to give the target physical inertia. For example: - State 1 = Constant Velocity (CV) - State 2 = Coordinated Turn (CT)

You might define your Π matrix as:

$$\Pi = \begin{bmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{bmatrix}$$

- Row 1 means: "If the drone is flying straight, there is a 95% chance it will keep flying straight, and a 5% chance it will initiate a turn."
- Row 2 means: "If the drone is actively turning, there is a 90% chance it will keep turning, and a 10% chance it will flatten out into straight flight."

2. The Interaction Step (Mixing)

Before the visual measurements even arrive, the IMM uses the Markov matrix to mix the tracking data. It calculates the Mixing Probabilities ($\mu_{i|j}$), which ask: "Given that we end up in Mode j, what is the probability we came from Mode i?"

$$\mu_{i|j} = \frac{1}{\bar{c}_j} p_{ij} \mu_i$$

Where μ_i is the current probability of the model from the previous frame, p_{ij} is the Markov transition scalar, and $\bar{c}_j$ is a normalizer.

The filter then calculates a "mixed" initial state vector and covariance matrix for every model. This prevents the CT model from losing track of the target while it sits idle during long periods of straight flight.

A. Calculate the Predicted Mode Probabilities ($\bar{c}_j$):

What is the prior probability of being in each mode today, based purely on yesterday's modes and our Markov matrix?

$$\bar{c}_1 = (p_{11} \times \mu_1) + (p_{21} \times \mu_2) = (0.95 \times 0.80) + (0.10 \times 0.20) = 0.76 + 0.02 = 0.78$$
$$\bar{c}_2 = (p_{12} \times \mu_1) + (p_{22} \times \mu_2) = (0.05 \times 0.80) + (0.90 \times 0.20) = 0.04 + 0.18 = 0.22$$

(Note: $0.78 + 0.22 = 1.0$. The math holds).

B. Calculate the Mixing Weights ($\mu_{i|j}$):

If we end up in Model 1 today, how much of yesterday's Model 1 and Model 2 should we mix into it?

$$\mu_{1|1} = (0.95 \times 0.80)/0.78 = 0.974$$
$$\mu_{2|1} = (0.10 \times 0.20)/0.78 = 0.026$$

C. Calculate the Mixed Initial States:

We blend the kinematic states together. The new starting velocity for Model 1 is heavily weighted by its own history, but it absorbs a tiny bit (2.6%) of Model 2's history.

$$\hat{x}_{01} = (0.974 \times 100) + (0.026 \times 90) = 99.74 \text{ m/s}$$

(The CKF Covariance matrices are mixed using similar weighted math, pooling their uncertainties together).

3. The Filtering Step (CKF)

The individual Cubature Kalman Filters now run independently. The CV model predicts a straight line; the CT model predicts a curve. Both generate cubature points and compare them to the incoming visual bounding box. Each filter calculates its own Likelihood (Λ_j)—essentially, how well its prediction matched reality.

Now, the two separate Cubature Kalman Filters run their standard prediction and update steps using the newly mixed initial states.

Let's assume the drone just executed a violent evasive turn.

- The CV Filter predicts the drone kept flying straight. It compares its prediction to the new camera bounding box. The error is massive. It outputs a tiny Measurement Likelihood: $\Lambda_1 = 0.01$.
- The CT Filter predicts a curve. It compares its prediction to the camera box. It's a very close match! It outputs a high Measurement Likelihood: $\Lambda_2 = 0.40$.

Let's say the filters output their newly calculated velocities as $\hat{x}_{new,1} = 95$ m/s and $\hat{x}_{new,2} = 85$ m/s.

4. The Model Probability Update

This is where Bayes' Theorem and the Markov process meet. The overall probability of each mode (μ_j) is updated by multiplying its Markov-predicted likelihood ($\bar{c}_j$) by its visual measurement likelihood (Λ_j):

$$\mu_j = \frac{1}{c}\Lambda_j \bar{c}_j$$

If the target executes a violent maneuver, the CV model's visual likelihood drops to zero. The CT model's likelihood spikes. The IMM instantly shifts the dominant probability μ to the CT model, and the final combined output trajectory curves perfectly with the target.

This is where Bayes' Theorem executes the mode switch. We update the overall probability of each model by multiplying the Markov Prediction ($\bar{c}$) by the Visual Likelihood (Λ).

A. Multiply Prior $\times$ Likelihood:

$$\mu_1^* = \Lambda_1 \times \bar{c}_1 = 0.01 \times 0.78 = 0.0078$$
$$\mu_2^* = \Lambda_2 \times \bar{c}_2 = 0.40 \times 0.22 = 0.0880$$

B. Normalize (Calculate Evidence c):

$$c = 0.0078 + 0.0880 = 0.0958$$

C. The Final Mode Probabilities:

$$\mu_1 = 0.0078/0.0958 = 0.081(8.1\%CV)$$
$$\mu_2 = 0.0880/0.0958 = 0.919(91.9\%CT)$$

The Engineering Result: In a single frame, the system mathematically realized the target was maneuvering. The Markov transition matrix allowed the probability to seamlessly violently flip from 80% CV to 91.9% CT without resetting the tracking pipeline

5. Estimate Combination

The final output of the IMM—the data actually sent to the weapons system or gimbal—is a weighted sum of the two filters, governed by our new mode probabilities.

$$\hat{x}_{final} = (\mu_1 \times \hat{x}_{new,1}) + (\mu_2 \times \hat{x}_{new,2})$$
$$\hat{x}_{final} = (0.081 \times 95) + (0.919 \times 85)$$
$$\hat{x}_{final} = 7.695 + 78.115 = 85.81 \text{ m/s}$$

Because the probability flipped to 91.9% CT, the final system output almost entirely trusts the Coordinated Turn model's estimate, perfectly adapting to the evasive maneuver.

C++ Implementation: IMM Markov Mixing

```cpp
#include <Eigen/Dense>
#include <iostream>

using namespace Eigen;

class MarkovProcess {
public:
    // Calculates the predicted mode probabilities based on the Markov Transition Matrix
    // current_probs: The likelihood of each mode from the previous frame
    // transition_matrix: The Pi matrix defining the maneuver inertia
    static VectorXd predictModeProbabilities(const VectorXd& current_probs,
                                             const MatrixXd& transition_matrix) {

        // Ensure inputs are valid
        if (current_probs.rows() != transition_matrix.rows()) {
            throw std::invalid_argument("Dimension mismatch in Markov mixing.");
        }

        // The Markov prediction is a simple matrix-vector multiplication
        // mu_predicted = Pi^T * mu_current
        VectorXd predicted_probs = transition_matrix.transpose() * current_probs;

        // Normalize to ensure total probability strictly equals 1.0
        double sum = predicted_probs.sum();
        if (sum > 0) {
            predicted_probs /= sum;
        }

        return predicted_probs;
    }
};
```

Part II: Dynamic Systems and The Optimal Estimator

Chapter 5

Control System Fundamentals

5.1 State-space representation of dynamic systems.

5.2 Observability and controllability.

5.3 Process noise vs. measurement noise.

Chapter 6

The Bayesian Filtering Framework

6.1 The recursive Bayes filter: Prediction and Update steps.

6.2 Why closed-form solutions are rare (the integration problem).

Part III: The Classic Standard Kalman Filter (KF)

Chapter 7

Derivation of the Standard KF

7.1 Mathematical proof of the linear Kalman Filter from Bayesian principles.

7.2 The Minimum Mean Square Error (MMSE) estimator.

Chapter 8

Tuning and Diagnostics

8.1 Covariance matching and innovation analysis.

8.2 Filter divergence and numerical stability considerations.

Part IV: Conquering Non-Linearity

Chapter 9

The Extended Kalman Filter (EKF)

9.1 Linearizing non-linear functions using the Jacobian.

9.2 Mathematical proof and limitations of the EKF (truncation errors).

Chapter 10

The Unscented Kalman Filter (UKF)

10.1 The Unscented Transform (UT): Choosing sigma points.

10.2 Proof of how UT captures mean and covariance better than linearization.

Chapter 11

The Cubature Kalman Filter (CKF)

11.1 The spherical-radial cubature rule.

11.2 Mathematical derivation of cubature points.

11.3 Comparative analysis: CKF vs. UKF in high-dimensional state spaces.

Part V: Maneuvering Targets and Multiple Models

Chapter 12

Interacting Multiple Model (IMM) Estimation

12.1 The challenge of maneuvering target tracking.

12.2 Markov chain mixing of model probabilities.

12.3 The IMM architecture: Interaction, Filtering, Model Probability Update, and Combination.

Chapter 13

The Synthesis: IMM-CKF

13.1 Embedding the Cubature Kalman Filter within the IMM framework.

13.2 Handling highly non-linear maneuvers (e.g., transitioning from a linear trajectory to a sharp, high-G coordinated turn).

13.3 Complete algorithm walkthrough and mathematical proof of convergence

Part VI: Architecture and Implementation

Chapter 14

Software Architecture for State Estimation

14.1 Object-oriented design patterns for filter development (e.g., in modern C++).

14.2 Memory management and real-time processing constraints.

Chapter 15

Hardware Deployment Considerations

15.1 Deploying complex estimators on embedded systems.

15.2 Exploiting parallelization for matrix operations.

Appendix

Chapter 16

The square root of a matrix and Cholesky decomposition

The square root of a matrix

A Matrix B is a square root of a matrix A if :

$$A = BB$$

The equation above can have several possible solutions, and there are different computation methods for finding the square root of a matrix.

However, in the context of state estimation, Uncentenced Kalman Filter(UKF) and the Cubature Kalman Filter (CKF), when we speak of the "square root" of a Covariance Matrix (P), we are usually referring to a specific spatial factorization. Because a covariance matrix is symmetric, we are looking for a matrix S such that:

$$P = SS^T$$

Where S^T is the transpose of S. While there are many ways to find an S that satisfies this equation (such as Singular Value Decomposition), the most computationally efficient and numerically stable method for tracking filters is the Cholesky Decomposition.

Positive and semi-definite Matrix Cholesky decoposition

Before we can decompose a matrix using the Cholesky algorithm, the matrix must satisfy a strict mathematical condition: it must be symmetric and positive definite (or positive semi-definite).

1. The Mathematical Definition

A symmetric $n \times n$ matrix A is Positive Definite if, for any non-zero real column vector x, the following holds true:

$$x^T Ax > 0$$

2. The Physical Tracking Definition

In target tracking, the matrix A represents our State Covariance Matrix (P). The diagonals of P are the variances of our state variables (position, velocity, etc.).

Because variance represents physical uncertainty (standard deviation squared, σ^2), it is physically impossible to have "negative uncertainty." Therefore, a valid covariance matrix will always be positive semi-definite. If numerical floating-point errors cause an eigenvalue to slip below zero, the matrix loses its positive definiteness, and algorithms attempting to find its square root will fail mathematically, returning imaginary numbers (NaNs in C++).

16.1 Theorem: Cholesky Decomposition Proof and Derivation**

Let A be an $n \times n$ symmetric, positive-definite matrix. There exists a unique lower triangular matrix L with strictly positive diagonal entries ($l_{ii} > 0$) such that

$$A = LL^T$$

A lower triangular matrix is simply a matrix where all entries above the main diagonal are zero.Now we use mathematical induction to proof this Theorem.

1. The Base Case ($n = 1$)

Let A be a 1×1 matrix, so $A = [a_{11}]$. Because A is positive-definite, for any non-zero vector x (which in this case is a scalar $x \neq 0$), $x^T Ax > 0$.

If we set $x = 1$, then $1 \cdot a_{11} \cdot 1 = a_{11} > 0$. We seek a 1×1 lower triangular matrix $L = [l_{11}]$ such that $A = LL^T$.

$$[a_{11}] = [l_{11}][l_{11}] \implies a_{11} = l_{11}^2$$

Since $a_{11} > 0$, there exists a unique, strictly positive real number $l_{11} = \sqrt{a_{11}}$. Thus, the theorem holds for $n = 1$.

2. The Inductive Hypothesis

Assume that the theorem holds for all symmetric, positive-definite matrices of size $k \times k$. This means any $k \times k$ positive-definite matrix can be uniquely decomposed into a lower triangular matrix times its transpose.

3. The Inductive Step ($n = k + 1$)

Now consider a $(k + 1) \times (k + 1)$ symmetric, positive-definite matrix A. We partition A into smaller block matrices:

$$A = \begin{bmatrix} A_{11} & a_{12} \\ a_{12}^T & a_{22} \end{bmatrix}$$

Where: - A_{11} is a $k \times k$ matrix. - a_{12} is a $k \times 1$ column vector. - a_{22} is a scalar (a 1×1 matrix).

Step A: Establishing A_{11} is Positive-Definite

Because A is symmetric and positive-definite, any principal submatrix of A must also be symmetric and positive-definite. Therefore, A_{11} is symmetric and positive-definite.

By our inductive hypothesis, because A_{11} is a $k \times k$ positive-definite matrix, there exists a unique $k \times k$ lower triangular matrix L_{11} with positive diagonals such that:

$$A_{11} = L_{11} L_{11}^T$$

Step B: Constructing L

We want to find a $(k+1) \times (k+1)$ lower triangular matrix L partitioned in the same way as A:

$$L = \begin{bmatrix} L_{11} & 0 \\ l_{21}^T & l_{22} \end{bmatrix}$$

Where:

- L_{11} is the $k \times k$ matrix from our inductive hypothesis.
- l_{21} is a $k \times 1$ column vector.
- l_{22} is a strictly positive scalar.

Let's compute LL^T:

$$LL^T = \begin{bmatrix} L_{11} & 0 \\ l_{21}^T & l_{22} \end{bmatrix} \begin{bmatrix} L_{11}^T & l_{21} \\ 0 & l_{22} \end{bmatrix} = \begin{bmatrix} L_{11}L_{11}^T & L_{11}l_{21} \\ l_{21}^T L_{11}^T & l_{21}^T l_{21} + l_{22}^2 \end{bmatrix}$$

Step C: Equating LL^T to A

By setting LL^T equal to our partitioned A, we get a system of equations:

1 $L_{11}L_{11}^T = A_{11}$
(This is already satisfied by our inductive hypothesis).

2 $L_{11}l_{21} = a_{12}$

Because L_{11} is a lower triangular matrix with strictly positive diagonal entries, its determinant (the product of its diagonals) is non-zero. This means L_{11} is invertible. Therefore, we can uniquely solve for the vector l_{21}:

$$l_{21} = L_{11}^{-1}a_{12}$$

3 $l_{21}^T l_{21} + l_{22}^2 = a_{22}$ We need to solve for the scalar l_{22}:

$$l_{22}^2 = a_{22} - l_{21}^T l_{21}$$

Step D: Proving l_{22} Exists and is Unique

For l_{22} to be a unique, strictly positive real number, we must prove that $(a_{22} - l_{21}^T l_{21}) > 0$.

Let's construct a specific $(k+1) \times 1$ test vector x:

$$x = \begin{bmatrix} y \\ -1 \end{bmatrix}$$

Where y is defined as $y = A_{11}^{-1}a_{12}$.

Because A is positive-definite, $x^T Ax > 0$ for any non-zero vector x. Let's evaluate this:

$$x^T Ax = \begin{bmatrix} y^T & -1 \end{bmatrix} \begin{bmatrix} A_{11} & a_{12} \\ a_{12}^T & a_{22} \end{bmatrix} \begin{bmatrix} y \\ -1 \end{bmatrix}$$

First, multiply the matrix A by the vector x:

$$\begin{bmatrix} A_{11} & a_{12} \\ a_{12}^T & a_{22} \end{bmatrix} \begin{bmatrix} y \\ -1 \end{bmatrix} = \begin{bmatrix} A_{11}y - a_{12} \\ a_{12}^T y - a_{22} \end{bmatrix}$$

Substitute $y = A_{11}^{-1}a_{12}$ into the top row:

$$A_{11}(A_{11}^{-1}a_{12}) - a_{12} = a_{12} - a_{12} = 0$$

Now multiply by x^T:

$$x^T Ax = \begin{bmatrix} y^T & -1 \end{bmatrix} \begin{bmatrix} 0 \\ a_{12}^T y - a_{22} \end{bmatrix} = -1 \cdot (a_{12}^T y - a_{22}) = a_{22} - a_{12}^T y$$

Substitute $y = A_{11}^{-1}a_{12}$ back into this result:

$$x^T Ax = a_{22} - a_{12}^T A_{11}^{-1} a_{12}$$

Since $x^T Ax > 0$, we now know that $a_{22} - a_{12}^T A_{11}^{-1} a_{12} > 0$.

Finally, let's look back at our term $l_{21}^T l_{21}$ from Step C. Recall that $l_{21} = L_{11}^{-1}a_{12}$.

$$l_{21}^T l_{21} = (L_{11}^{-1}a_{12})^T (L_{11}^{-1}a_{12}) = a_{12}^T (L_{11}^{-1})^T L_{11}^{-1} a_{12}$$

Since $(L_{11}^{-1})^T L_{11}^{-1} = (L_{11}L_{11}^T)^{-1} = A_{11}^{-1}$, we get:

$$l_{21}^T l_{21} = a_{12}^T A_{11}^{-1} a_{12}$$

Therefore, substituting this back into our equation for l_{22}^2:

$$l_{22}^2 = a_{22} - a_{12}^T A_{11}^{-1} a_{12}$$

Since we just proved that this exact expression is strictly greater than zero, $l_{22}^2 > 0$. Thus, $l_{22} = \sqrt{a_{22} - l_{21}^T l_{21}}$ exists and is a unique, strictly positive real number.

Conclusion

By the principle of mathematical induction, because the theorem holds for $n = 1$, and assuming it holds for k guarantees it holds for $k + 1$, we conclude that every symmetric, positive-definite matrix can be uniquely factored into $A = LL^T$.

16.2 Algorithm: Cholesky Decomposition

A lower triangular matrix is simply a matrix where all entries above the main diagonal are zero. To showcase how this factorization is deterministically achieved, let us manually expand the equation for a 3×3 matrix.Let the known symmetric covariance matrix be A:

$$A = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{21} & a_{22} & a_{32} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

(Note: Because A is symmetric, $a_{12} = a_{21}$, $a_{13} = a_{31}$, etc.)

We want to find the unknown lower triangular matrix L:

$$L = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix}$$

And its transpose L^T:

$$L^T = \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

If we physically multiply $L \times L^T$, we get:

$$LL^T = \begin{bmatrix} l_{11}^2 & l_{11}l_{21} & l_{11}l_{31} \\ l_{21}l_{11} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} \\ l_{31}l_{11} & l_{31}l_{21} + l_{32}l_{22} & l_{31}^2 + l_{32}^2 + l_{33}^2 \end{bmatrix}$$

Because $A = LL^T$, we can now set the elements of our multiplied matrix directly equal to the elements of A and solve for the unknown l values recursively, starting from the top-left corner.

Step 1: Solve the first column

$$a_{11} = l_{11}^2 \implies l_{11} = \sqrt{a_{11}}$$
$$a_{21} = l_{21}l_{11} \implies l_{21} = \frac{a_{21}}{l_{11}}$$
$$a_{31} = l_{31}l_{11} \implies l_{31} = \frac{a_{31}}{l_{11}}$$

Step 2: Solve the seconde column

$$a_{22} = l_{21}^2 + l_{22}^2 \implies l_{22} = \sqrt{a_{22} - l_{21}^2}$$
$$a_{32} = l_{21}l_{31} + l_{22}l_{32} \implies l_{32} = \frac{a_{32} - l_{21}l_{31}}{l_{22}}$$

Step 3: Solve the third column

$$a_{33} = l_{31}^2 + l_{32}^2 + l_{33}^2 \implies l_{33} = \sqrt{a_{33} - l_{31}^2 - l_{32}^2}$$

The General Algorithm

By recognizing the pattern in the algebraic proof above, we can abstract this into a set of generalized equations that a computer can run for an $n \times n$ matrix.For the diagonal elements ($j = i$): For the diagonal elements ($j = i$):

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$$

For the off-diagonal elements ($i > j$):

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right)$$

Why Cholesky for the IMM-CKF?

In the Cubature Kalman Filter, we must generate $2n$ cubature points at every single time step for every single active IMM model. To do this, we need the matrix "square root" of the predicted state covariance matrix P.

Using the Cholesky decomposition to find $P = LL^T$ is the premier choice for embedded edge engineering for two reasons:

- **Computational Efficiency**: Standard matrix square root algorithms (like SVD or eigenvalue decomposition) have high computational overhead. Because Cholesky exploits the inherent symmetry of the covariance matrix and only solves for half the matrix (the lower triangle), it requires approximately half the floating-point operations ($O(n^3/3)$).
- **Deterministic Point Generation**: Multiplying our standard unit vectors by the lower triangular matrix L cleanly scales and rotates our cubature points directly along the principal axes of the target's uncertainty ellipse, ensuring mathematically stable propagation through non-linear measurement models.

Chapter 17

Expectation Algebra and Covariance Propagation

To understand why the standard Kalman filter equations take the shape they do, one must understand how expectation (the expected value) acts as a mathematical operator.

1. Linearity of Expectation

The Expectation operator $E[\cdot]$ is perfectly linear. For any random variable x, constant matrix A, and constant vector B:

$$E[A\mathrm{x} + B] = AE[\mathrm{x}] + B$$

2. Definition of Covariance

The covariance matrix P of a random variable x with mean $\mu = E[\mathrm{x}]$ is defined as the expected value of the outer product of its deviations:

$$Cov(\mathrm{x}) = P = E[(\mathrm{x} - \mu)(\mathrm{x} - \mu)^T]$$

3. The Linear Transformation Proof

In tracking, our kinematic equations (like the Constant Velocity model) apply a linear transformation to our state vector to predict the future:

$$\mathrm{x}_{k+1} = F\mathrm{x}_k + \mathrm{w}$$

Where F is the transition matrix and w is zero-mean process noise ($E[\mathrm{w}] = 0$) with a covariance of Q.

Theorem: If a random variable x undergoes a linear transformation $\mathrm{y} = F\mathrm{x} + \mathrm{w}$, the covariance of y is $FPF^T + Q$.

Proof:

First, find the mean of our new variable y. Let the mean of x be μ_x.

$$\mu_y = E[\mathrm{y}] = E[F\mathrm{x} + \mathrm{w}]$$

By linearity of expectation:

$$\mu_y = FE[\mathrm{x}] + E[\mathrm{w}] = F\mu_x + 0$$

Now, apply the definition of covariance to y:

$$Cov(\mathrm{y}) = E[(\mathrm{y} - \mu_y)(\mathrm{y} - \mu_y)^T]$$

Substitute the definitions of y and μ_y:

$$Cov(\mathrm{y}) = E[(F\mathrm{x} + \mathrm{w} - F\mu_x)(F\mathrm{x} + \mathrm{w} - F\mu_x)^T]$$
$$Cov(\mathrm{y}) = E[(F(\mathrm{x} - \mu_x) + \mathrm{w})(F(\mathrm{x} - \mu_x) + \mathrm{w})^T]$$

Expand the outer product $(a+b)(a+b)^T = aa^T + ab^T + ba^T + bb^T$:

$$Cov(\mathrm{y}) = E[F(\mathrm{x} - \mu_x)(\mathrm{x} - \mu_x)^T F^T \quad + \quad F(\mathrm{x} - \mu_x)\mathrm{w}^T \quad + \quad \mathrm{w}(\mathrm{x} - \mu_x)^T F^T \quad + \quad \mathrm{w}\mathrm{w}^T]$$

Apply the expectation operator linearly across the terms. Because the state estimate error $(\mathrm{x} - \mu_x)$ and the random process noise w are statistically independent, the expectation of their cross-products is zero: $E[F(\mathrm{x} - \mu_x)\mathrm{w}^T] = 0$.This leaves:

$$Cov(\mathrm{y}) = F \cdot E[(\mathrm{x} - \mu_x)(\mathrm{x} - \mu_x)^T] \cdot F^T + E[\mathrm{w}\mathrm{w}^T]$$

Recognizing the original definitions of P and Q:

$$Cov(\mathrm{y}) = FPF^T + Q$$

This proof forms the mathematical basis for the Prediction Step in every linear and extended Kalman Filter ever written.

Chapter 18

The Gaussian Multiplication Proof and the Origins of the Kalman Gain

In Chapter 4, we stated that Bayes' Theorem operates by multiplying the Prior probability distribution (our kinematic prediction) by the Likelihood distribution (our sensor measurement). We also stated that because both of these are Gaussian (Normal) distributions, multiplying them magically produces a third, narrower Gaussian distribution representing our updated estimate (the Posterior).

But why does multiplying two bell curves create another perfect bell curve? And more importantly for an engineer, how does this algebra actually translate into code?

By stepping through the algebraic proof of multiplying two 1D Gaussian distributions, we will witness the exact mathematical birth of the Kalman Gain (K) and the standard update equations used in every tracking filter.

1. Defining the Two Distributions

Let us define our two independent pieces of information as 1D Gaussian Probability Density Functions (PDFs):

A. The Prior (Our Physics Prediction)

- Mean (State Estimate): μ_1
- Variance (Uncertainty): σ_1^2

$$p_{prior}(x) = \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma_1^2}\right)$$

B. The Likelihood (Our Sensor Measurement)

- Mean (Measured Value): μ_2
- Variance (Sensor Noise): σ_2^2

$$p_{likelihood}(x) = \frac{1}{\sqrt{2\pi\sigma_2^2}} \exp\left(-\frac{(x-\mu_2)^2}{2\sigma_2^2}\right)$$

2. Multiplying the Distributions

According to Bayes' theorem, to find our Posterior distribution, we multiply these two PDFs together.

$$p_{posterior}(x) \propto p_{prior}(x) \cdot p_{likelihood}(x)$$

When multiplying exponential functions, we simply add their exponents. We can safely ignore the scaling constants ($\frac{1}{\sqrt{2\pi\sigma^2}}$) for now, as they only serve to ensure the final curve's area equals 1.0. Let's focus entirely on the new exponent:

$$\exp\left(-\frac{(x-\mu_1)^2}{2\sigma_1^2}\right) \cdot \exp\left(-\frac{(x-\mu_2)^2}{2\sigma_2^2}\right) = \exp\left(-\frac{1}{2}\left[\frac{(x-\mu_1)^2}{\sigma_1^2} + \frac{(x-\mu_2)^2}{\sigma_2^2}\right]\right)$$

Now, let's group these terms by the powers of x (the x^2 terms, the x terms, and the constants):

$$x^2\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right) - 2x\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}\right) + \left(\frac{\mu_1^2}{\sigma_1^2} + \frac{\mu_2^2}{\sigma_2^2}\right)$$

3. Expanding the Exponent

To prove this results in a new Gaussian, we need to manipulate the bracketed sum into the standard Gaussian form: $\frac{(x-\mu_{new})^2}{\sigma_{new}^2}$.

Let's expand the quadratic terms inside the brackets:

$$\frac{x^2 - 2\mu_1 x + \mu_1^2}{\sigma_1^2} + \frac{x^2 - 2\mu_2 x + \mu_2^2}{\sigma_2^2}$$

Now, let's group these terms by the powers of x (the x^2 terms, the x terms, and the constants):

$$x^2\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right) - 2x\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}\right) + \left(\frac{\mu_1^2}{\sigma_1^2} + \frac{\mu_2^2}{\sigma_2^2}\right)$$

4. Completing the Square (Finding the New Mean and Variance)

We want our grouped equation to perfectly match the expanded form of our target Gaussian exponent:

$$\frac{1}{\sigma_{new}^2}(x^2 - 2\mu_{new}x + \mu_{new}^2)$$

By matching the coefficients of x^2, we can find our new variance (σ_{new}^2):

$$\frac{1}{\sigma_{new}^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}$$

Solving for σ_{new}^2:

$$\sigma_{new}^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

Next, by matching the coefficients of x, we can find our new mean (μ_{new}):

$$\frac{\mu_{new}}{\sigma_{new}^2} = \frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}$$

Substitute our newly found σ_{new}^2 and solve for μ_{new}:

$$\mu_{new} = \left(\frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}\right)\left(\frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2\sigma_2^2}\right)$$

The cross-terms cancel out, leaving:

$$\mu_{new} = \frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

We have now mathematically proven that multiplying two Gaussians creates a new Gaussian, and we have the exact formulas for its Mean and Variance.

5. The Engineering Translation: The Birth of the Kalman Gain

While the formulas above are mathematically pure, they are not how engineers write tracking code. Let's rewrite our new Mean (μ_{new}) formula by splitting it apart and rearranging it:

$$\mu_{new} = \mu_1\frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} + \mu_2\frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

Through basic algebra, we can factor this into an "update" format—taking our original prediction (μ_1) and adding a correction to it:

$$\mu_{new} = \mu_1 + \left(\frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}\right)(\mu_2 - \mu_1)$$

Let's look at this equation through the lens of a tracking engineer:

- μ_1 is our Predicted State ($\hat{x}_{k|k-1}$)
- μ_2 is our Sensor Measurement (z_k)
- $(\mu_2 - \mu_1)$ is our Prediction Error, or Innovation (y_k).

What is that fractional term in the middle? That is the ratio of our prediction uncertainty to the total system uncertainty. It dictates exactly how much we should trust the sensor's innovation.

This is the 1D Kalman Gain (K):

$$K = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

Substituting K back into our equations yields the exact, universally recognized 1D Kalman Filter update equations:

- State Update: $\hat{x}_{k|k} = \hat{x}_{k|k-1} + K(z_k - \hat{x}_{k|k-1})$

- Covariance Update: $\sigma_{new}^2 = (1 - K)\sigma_1^2$

The Matrix Generalization

When scaling from a 1D example to the multi-dimensional tracking of a drone using an IMM-CKF, the algebra remains identical, but the notation shifts to linear algebra matrices.

- The Prior Variance (σ_1^2) becomes the Predicted Covariance Matrix (P).
- The Likelihood Variance (σ_2^2) becomes the Measurement Noise Matrix (R).
- The addition in the denominator becomes a matrix inversion.

Thus, the multidimensional Kalman Gain emerges directly from the algebra of Gaussian multiplication:

$$K = PH^T(HPH^T + R)^{-1}$$

(Note: The H matrix simply projects the state space into the measurement space so the matrices align properly).

Chapter 19

Bibliography

19.1 Articles

[1] Arasaratnam, I., & Haykin, S. (2009). *Cubature Kalman Filters*. IEEE Transactions on Automatic Control.

19.2 Books

[2] Simon, D. (2006). *Optimal State Estimation: Kalman, H∞, and Nonlinear Approaches*. John Wiley & Sons.

Bibliography

Index